

We define subsequence as any subset of an array. We define a subarray as a contiguous subsequence in an array.

Given an array, find the maximum possible sum among:

1. all nonempty subarrays.
2. all nonempty subsequences.

Print the two values as space-separated integers on one line.

Note that empty subarrays/subsequences should not be considered.

Example

$arr = [-1, 2, 3, -4, 5, 10]$

The maximum subarray sum is comprised of elements at indices $[1 - 5]$. Their sum is $2 + 3 + -4 + 5 + 10 = 16$. The maximum subsequence sum is comprised of elements at indices $[1, 2, 4, 5]$ and their sum is $2 + 3 + 5 + 10 = 20$.

Function Description

Complete the `maxSubarray` function in the editor below.

`maxSubarray` has the following parameter(s):

- `int arr[n]`: an array of integers

Returns

- `int[2]`: the maximum subarray and subsequence sums

```

20
21     int current = arr.get(0);
22     int maxSubarray = arr.get(0);
23
24     int maxSubsequence = 0;
25     int maxElement = arr.get(0);
26
27     for (int i = 0; i < arr.size(); i++) {
28
29         int num = arr.get(i);
30
31         // Maximum subarray
32         if (i > 0) {
33             current = Math.max(num, current + num);
34             maxSubarray = Math.max(maxSubarray, current);
35         }
36
37         // Maximum subsequence

```

Line: 56 Col: 6

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test



Search

ENG
IN

12:14
27-08-2026

Problem

Submissions

Leaderboard

Two children, Lily and Ron, want to share a chocolate bar. Each of the squares has an integer on it.

Lily decides to share a contiguous segment of the bar selected such that:

- The length of the segment matches Ron's birth month, and,
- The sum of the integers on the squares is equal to his birth day.

Determine how many ways she can divide the chocolate.

Example

$s = [2, 2, 1, 3, 2]$

$d = 4$

$m = 2$

Lily wants to find segments summing to Ron's birth day, $d = 4$ with a length equalling his birth month, $m = 2$. In this case, there are two segments meeting her criteria: $[2, 2]$ and $[1, 3]$.

Function Description

Complete the birthday function in the editor below.

birthday has the following parameter(s):

- int $s[n]$: the numbers on each of the squares of chocolate
- int d : Ron's birth day
- int m : Ron's birth month

```
19  */
20
21  public static int birthday(List<Integer> s, int d, int m) {
22
23      int count = 0;
24      int sum = 0;
25
26      // Calculate sum of the first window of size m
27      for (int i = 0; i < m; i++) {
28          sum += s.get(i);
29      }
30
31      if (sum == d) {
32          count++;
33      }
34
35      // Slide the window across the list
36      for (int i = m; i < s.size(); i++) {
```

Line: 79 Col: 6



Upload Code as File



Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

You are given a string containing characters *A* and *B* only. Your task is to change it into a string such that there are no matching adjacent characters. To do this, you are allowed to delete zero or more characters in the string.

Your task is to find the minimum number of required deletions.

Example

s = *AABAAB*

Remove an *A* at positions **0** and **3** to make *s* = *ABAB* in **2** deletions.

Function Description

Complete the `alternatingCharacters` function in the editor below.

`alternatingCharacters` has the following parameter(s):

- string *s*: a string

Returns

- int: the minimum number of deletions required

Input Format

The first line contains an integer *q*, the number of queries.

The next *q* lines each contain a string *s* to analyze.

Constraints

- $1 \leq q \leq 10$

```
16  */
17
18  public static int alternatingCharacters(String s) {
19
20      int deletions = 0;
21
22      for (int i = 1; i < s.length(); i++) {
23          if (s.charAt(i) == s.charAt(i - 1)) {
24              deletions++;
25          }
26      }
27
28      return deletions;
29  }
30
31
32 }
```

Line: 19 Col: 5

 Upload Code as File☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

 Search...

Courses ▾Tutorials ▾Practice ▾Jobs ▾

≡

Problem

Editorial

Submissions

Comments

Input: txt = "abesdu", pat = "edu"

Output: []

Explanation: There's no substring "edu" present in txt.

Input: txt = "aabaacaadaabaaba", pat = "aaba"

Output: [0, 9, 12]

Explanation:
Text: a a b a a c a a d a a b a a b a
Pattern: a a b a

Output Window

Compilation ResultsCustom Input

Compilation Completed

Case 1

Java (21)

Start Timer

```
27
28
29 // KMP search
30 i = 0; // index for txt
31 int j = 0; // index for pat
32
33 while (i < n) {
34     if (txt.charAt(i) == pat.charAt(j)) {
35         i++;
36         j++;
37     }
38
39     if (j == m) {
40         result.add(i - j);
41
42         // Continue searching for overlapping occurrences
43         j = lps[j - 1];
44     } else if (i < n && txt.charAt(i) != pat.charAt(j)) {
45         if (j > 0) {
46             j = lps[j - 1];
47         } else {
48             i++;
49         }
50     }
51 }
52
53 return result;
54
55 }
56
```

Custom InputCompile & RunSubmit

Problem List

Accepted

Editorial

Solutions

Submissions

1408. String Matching in an Array

Solved

Easy

Topics

Companies

Hint

Given an array of string `words`, return all strings in `words` that are a **substring** of another word. You can return the answer in **any order**.

Example 1:

Input: `words = ["mass","as","hero","superhero"]`
Output: `["as","hero"]`
Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".
["hero","as"] is also a valid answer.

Example 2:

Input: `words = ["leetcode","et","code"]`
Output: `["et","code"]`
Explanation: "et", "code" are substring of "leetcode".

1.5K 173

Browse right away. Launch Edge when Windows starts. Turn on

Submit

Code

Java Auto

```
1 import java.util.*;
2
3 class Solution {
4     public List<String> stringMatching(String[] words) {
5         List<String> result = new ArrayList<>();
6
7         for (int i = 0; i < words.length; i++) {
8             for (int j = 0; j < words.length; j++) {
9                 if (i != j && words[j].contains(words[i])) {
10                     result.add(words[i]);
11                     break;
12                 }
13             }
14         }
15     }
16 }
```

Saved Ln 19, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Problem List

Submit

Premium

Description

Accepted

Editorial

Solutions

Submissions

890. Find and Replace Pattern

Solved

Medium

Topics

Companies

Given a list of strings `words` and a string `pattern`, return a list of `words[i]` that match `pattern`. You may return the answer in **any order**.

A word matches the pattern if there exists a permutation of letters `p` so that after replacing every letter `x` in the pattern with `p(x)`, we get the desired word.

Recall that a permutation of letters is a bijection from letters to letters: every letter maps to another letter, and no two letters map to the same letter.

Example 1:

Input: `words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"`
Output: `["mee","aqq"]`
Explanation: "mee" matches the pattern because there is a permutation {a -> m, b -> e, ...}.
"ccc" does not match the pattern because {a -> c, b -> c, ...} is not a permutation.

4K

28

☆

🔖

🔍

</> Code

Java

Auto

```
30
31
32 // Check word -> pattern mapping
33 if (reverseMap.containsKey(w) && reverseMap.get(w) != p) {
34     return false;
35 }
36
37 map.put(p, w);
38 reverseMap.put(w, p);
39
40
41 return true;
42
43
44
```

Saved

Ln 44, Col 1

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Problem List

Submit

0

Premium

Description

Editorial

Solutions

Submissions

3. Longest Substring Without Repeating Characters

Medium Topics Companies Hint

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

Input: `s = "abcabcbb"`
Output: `3`
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: `s = "bbbb"`
Output: `1`
Explanation: The answer is "b", with the length of 1.

Example 3:

45.6K 939

</> Code

Java Auto

```
1 class Solution {
2     public int lengthOfLongestSubstring(String s) {
3         int[] lastIndex = new int[128];
4         java.util.Arrays.fill(lastIndex, -1);
5
6         int left = 0;
7         int maxLength = 0;
8
9         for (int right = 0; right < s.length(); right++) {
10             char ch = s.charAt(right);
11
12             // Move left pointer if character was already seen
13             if (lastIndex[ch] >= left) {
14                 left = lastIndex[ch] + 1;
15             }
16         }
17     }
18 }
```

Saved Ln 25, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Problem List

8. String to Integer (atoi)

Medium Topics Companies

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").

2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.

3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.

4. **Rounding:** If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

6.4K 992

Solved

Code

Java Auto

```
1 class Solution {
2     public int myAtoi(String s) {
3         int i = 0;
4         int n = s.length();
5
6         // 1. Skip leading whitespace
7         while (i < n && s.charAt(i) == ' ') {
8             i++;
9         }
10
11        // 2. Check sign
12        int sign = 1;
13
14        if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
15            if (s.charAt(i) == '-') {
```

Saved Ln 39, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Case 4

Case 5

Solved

Given a **circular integer array** `nums` of length `n`, return the maximum possible sum of a non-empty **subarray** of `nums`.

A **subarray** may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist $1 \leq k_1, k_2 \leq j$ with $k_1 \neq k_2$ and $nums[k_1] = nums[k_2]$.

Input: nums = [1,-2,3,-2]

Output: 3

Explanation: Subarray [3] has maximum sum 3.

Java Auto

20				COLUMBIA - 11/11/11
----	--	--	--	---------------------

```

21     }
22
23     // If all elements are negative, circular sum would be 0,
24     // which is invalid because the subarray must be non-empty.
25     if (maxSum < 0) {
26         return maxSum;
27     }
28
29     // Maximum circular subarray =
30     // total sum - minimum subarray
31     int circularMax = totalSum - minSum;
32
33     return Math.max(maxSum, circularMax);
34 }
35 }

```

Saved

Ln 36, Col 1

Testcase > Test Result

Accepted Runtime: 0 ms

✔ Case 1 ✔ Case 2 ✔ Case 3

Problem List
<
>
↺
Submit
📄
✦
🔍
⚙️
🔥 0
🕒
👤
Premium

Description
Accepted
Editorial
Solutions
Submissions

53. Maximum Subarray

Medium Topics Companies

Given an integer array `nums`, find the [subarray](#) with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`

38.3K 583 ☆ 📌 ?

Solved

Code

Java Auto

```

1 class Solution {
2     public int maxSubArray(int[] nums) {
3         int currentSum = nums[0];
4         int maxSum = nums[0];
5
6         for (int i = 1; i < nums.length; i++) {
7             currentSum = Math.max(nums[i], currentSum + nums[i]);
8             maxSum = Math.max(maxSum, currentSum);
9         }
10
11         return maxSum;
12     }
13 }
```

Saved Ln 14, Col 1

☒ Testcase > **Test Result**

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input